Architecture strategies for optimizing scaling costs

Applies to this Azure Well-Architected Framework Cost Optimization checklist recommendation:

Expand table

CO:12	**Optimize scaling costs. Evaluate alternative scaling within your scale units. Consider alternative scaling configurations, and align with the cost model. Considerations should include utilization against the inherit limits of every instance, resource, and scale unit boundary. Use strategies for controlling demand and supply.**

This guide provides recommendations for optimizing scaling costs. Cost optimizing scaling is the process of removing inefficiencies in workload scaling. The goal is to reduce scaling costs while still meeting all nonfunctional requirements. Spending less to get the same result. Optimizing scaling allows you to avoid unnecessary expenses, overprovisioning, and waste. It also helps prevent unexpected spikes in costs by controlling demand and capping supply. Inefficient scaling practices can lead to increased workload and operational costs and negatively affect the overall financial health of the workload.

Definitions

Expand table

Term	Definition
Autoscaling	A scaling approach that automatically adds or removes resources when a set of conditions is met.
Cost metrics	Numeric data related to workload cost.
Scale down	A vertical scaling strategy that shifts to a lower SKU to provide less resources to the workload.
Scale in	A horizontal scaling strategy that removes instances to provide less resources to the workload.
Scale out	A horizontal scaling strategy that adds instances to provide more resources to the workload.
Scale unit	A group of resources that scale proportionately together.

Term	Definition
Scale up	A vertical scaling strategy that shifts to a higher SKU to provide more resources to the workload.
Stock keeping unit (SKU)	A service tier for an Azure service.
Usage data	Usage data is either direct information (real) or indirect/representative information (proxy) about how much a task, service, or application is being used.

The goal of cost optimizing scaling is to scale up and out at the last responsible moment and to scale down and in as soon as it's practical. To optimize scaling for your workload, you can evaluate alternative scaling options within the scale units and align them with the cost model. A scale unit represents a specific grouping of resources that can be scaled independently or together. You should design scale units to handle a specific amount of load, and they can comprise multiple instances, servers, or other resources. You need to evaluate the cost effectiveness of your workload scale units and model alternates.

If you don't use scaling, see guidance on scaling the workload. You need to figure out if your application can scale. Stateless applications are easier to scale because they can handle multiple requests at the same time. Also, evaluate if the application is built using distributed systems principles. Distributed systems can handle increased load by distributing the workload across multiple nodes. However, a singleton application is designed to have only one instance running at any given time. So scaling might not be appropriate for all workloads.

Evaluate scale out versus scale up

Evaluating scale out versus scale up involves determining the most cost-effective approach between increasing resources in an existing system (scale up) or adding more instances of that system (scale out) based on various factors like pricing, workload requirements, and acceptable downtime. Choosing the right scaling approach can lead to significant savings, ensuring you pay for only what you need while still meeting performance and reliability standards.

The goal is to determine the most cost-efficient choice based on service-tier pricing, workload traits, acceptable downtime, and the cost model. For some, it might be more economical to opt for more expensive instances in fewer numbers. Conversely, for others, a cheaper tier with more instances might be better. To make an informed decision, you need to analyze real or representative data from your setup and evaluate the relative cost merits of each strategy. To evaluate the most cost efficient approach, consider these recommendations:

- *Gather usage data*: Collect actual production data or proxy data that represents the workload usage patterns and resource utilization. This data should include metrics such as CPU usage, memory usage, network traffic, and any other relevant metrics that affect the cost of scaling.

- *Define cost metrics*: Identify the cost metrics that are relevant to your workload, such as the cost per hour, cost per transaction, or cost per unit of resource usage. These metrics help you compare the cost effectiveness of different scaling options.

- *Gather usage data*: Collect actual production data or proxy data that represents the workload usage patterns and resource utilization. This data should include metrics such as CPU usage, memory usage, network traffic, and any other relevant metrics that affect the cost of scaling

- *Define cost metrics*: Identify the cost metrics that are relevant to your workload, such as the cost per hour, cost per transaction, or cost per unit of resource usage. These metrics help you compare the cost-effectiveness of different scaling options.

- *Refer to requirements*: When deciding between scale-out and scale-up strategies, consider the reliability, performance, and scaling requirements of your workload. Scaling out can improve reliability through redundancy. Scaling up increases the capacity of a resource, but there might be limits to how much you can scale up.

- *Consider resource limits*: When evaluating scaling options, it's important to consider the inherent limits of every instance, resource, and scale unit boundary. Be aware of the upper scaling limits for each resource and plan accordingly. Additionally, keep in mind the limits of your subscription and other resources.

- *Test scaling*: Create tests for different scaling scenarios, including scale out and scale up options. Applying the usage data, simulate the workload behavior under different scaling configurations. Conduct real-world testing using the modeled scaling scenarios.

- *Calculate costs*: Use the gathered data and cost metrics to calculate the costs associated with each scaling configuration. Consider factors such as instance pricing, resource utilization, and any extra costs related to scaling.

Optimize autoscaling

Optimizing the autoscaling policy involves refining autoscaling to react to load changes based on the workload's nonfunctional requirements. You can limit excessive scaling activities by adjusting

thresholds and using the right cooldown period. To optimize autoscaling, consider the following recommendations:

- *Analyze the current autoscaling policy*: Understand the existing policy and its behavior in response to varying load levels.
- *Refer to nonfunctional requirements*: Identify the specific nonfunctional requirements that you need to consider, such as response time, resource utilization, or cost.
- *Adjust scaling thresholds*: Adjust the scaling thresholds based on the workload characteristics and nonfunctional requirements. Set thresholds for scaling up or down based on factors like CPU utilization over time, network traffic, or queue length.
- *Adjust a cooldown period*: Adjust the cooldown period to prevent excessive scaling activities triggered by temporary load spikes. A cooldown period introduces a delay between scaling events, allowing the system to stabilize before further scaling actions.
- *Monitor and fine-tune*: Continuously monitor the system's behavior and performance. Analyze the scaling activities and adjust the policy as needed to optimize cost and meet the desired nonfunctional requirements.

> **Tradeoff**: Reducing the number of scaling events raises the chances of encountering issues related to scaling. It means you're eliminating the extra cushion or buffer that could help manage potential problems or delays from scaling.

Use event-based scaling

Event-driven autoscaling allows the application to dynamically adjust resources based on specific events or triggers rather than traditional metrics like CPU or memory utilization. For example, Kubernetes event-driven autoscaling (KEDA) can scale applications based on scalers such as the length of a Kafka topic. Precision helps prevent unnecessary scaling fluctuations and resource waste. A high level of precision ultimately optimizes costs. To use event-based scaling, follow these steps:

- *Choose an event source*: Determine the event source that triggers the scaling of your scale unit. A source can be a message queue, a streaming platform, or any other event-driven system.
- *Set up event ingestion*: Configure your application to consume events from the chosen event source. It typically involves establishing a connection, subscribing to the relevant

topics or queues, and processing the incoming events.

- *Implement scaling logic*: Write the logic that determines when and how your scale unit should scale based on the incoming events. This logic should consider factors such as the number of events, the rate of incoming events, or any other relevant metrics.
- *Integrate with scaling mechanisms*: Depending on your application's runtime environment, you can use different scaling mechanisms to adjust the resources allocated to the application.
- *Configure scaling rules*: Define the scaling rules that specify how your scale unit should scale in response to events. These rules can be based on thresholds, patterns, or any other criteria that align with your application's requirements. Scaling thresholds should relate to business metrics. For example, if you add two more instances, you can support 50 more users in shopping cart processing.
- *Test and monitor*: Validate the behavior of your event-based scaling implementation by testing it with different event scenarios. Monitor the scaling actions and ensure that the actions align with your expectations.

>  **Tradeoff** Configuring and fine-tuning event-based autoscaling can be complex, and improper configuration might lead to over-provisioning or under-provisioning of resources.

Optimize demand and supply

Control demand against your supply. On workloads where usage determines scaling, cost correlates with the scaling. To optimize the costs of scaling, you can minimize scaling spend. You can offload demand by distributing demand to other resources, or you can reduce demand by implementing priority queues, gateway offloading, buffering, and rate limiting. Both strategies can prevent undesired costs due to scaling and resource consumption. You can also control supply by capping the scaling limits. To optimize workload demand and supply, consider the following recommendations.

Offload demand

Offloading demand refers to the practice of distributing or transferring resource demand to other resources or services. You can use various technologies or strategies:

- *Caching*: Use caching to store frequently accessed data or content, reducing the load on your backend infrastructure. For example, use content delivery networks (CDNs) to cache and serve static content, reducing the need for scaling the backend. However, not every workload can cache data. Workloads that require up-to-date and real-time data, like trading or gaming workloads, shouldn't use a cache. The cached data would be old and irrelevant to the user.

  > **Tradeoff**. Caching might introduce challenges in terms of cache invalidation, consistency, and managing cache expiration. It's important to carefully design and implement caching strategies to avoid potential tradeoffs.

- *Content offloading*: Offload content to external services or platforms to reduce the workload on your infrastructure. For example, rather than store video files on your primary server, you can host these files in a separate storage service that's independent from your primary server. You can load these large files directly from the storage service. This approach frees up resources on your servers, allowing you to use a smaller server. It can be cheaper to store large files in a separate data store. You can use a CDN to improve performance.

- *Load balancing*: Distribute incoming requests across multiple servers using load balancing. Load balancing evenly distributes the workload and prevents any single server from becoming overwhelmed. Load balancers optimize resource utilization and improve the efficiency of your infrastructure.

- *Database offloading*: Reduce the load on your main application server by offloading database operations to a separate database server or a specialized service. For example, use a CDN for static content caching and a Redis cache for dynamic content (data from database) caching. Techniques like database sharding, read replicas, or using managed database services can also reduce the load.

  > **Tradeoff:** Offloading specific tasks to alternate resources helps reduce or avoid extra scaling and costs associated with scaling. However, it's important to consider the operational and maintenance challenges that might arise from offloading. Conducting a comprehensive cost-benefit analysis is crucial when selecting the most appropriate offloading techniques for your workload. This analysis ensures that the chosen method is both efficient and feasible in relation to the anticipated savings and operational complexities.

Reduce demand

Reducing resource demand means implementing strategies that help minimize resource utilization in a workload. Offloading demand shifts demand to other resources. Reducing demand decreases demand on the workload. Reducing demand allows you to avoid overprovisioning resources and paying for unused or underutilized capacity. You should use code-level design patterns to reduce the demand on workload resources. To reduce demand through design patterns, follow these steps:

- *Understand design patterns*: Familiarize yourself with various design patterns that promote resource optimization.
- *Analyze workload requirements*: Assess the specific requirements of your workload, including its expected demand patterns, peak loads, and resource needs.
- *Select appropriate design patterns*: Choose the design patterns that align with your workload's requirements and objectives. For example, if your workload experiences fluctuating demand, event-driven scaling and throttling patterns can help manage the workload by dynamically allocating resources. Apply the selected design patterns to your workload architecture. You might need to separate workload components, containerize applications, optimize storage utilization, and more.
- *Continuously monitor and optimize*: Regularly evaluate the effectiveness of the implemented design patterns and adjust as needed. Monitor resource usage, performance metrics, and cost optimization opportunities.

By following these steps and using appropriate design patterns, you can reduce resource demand, optimize costs, and ensure the efficient operation of their workloads.

Use these design patterns to reduce demand:

- *Cache aside*: The pattern checks the cache to see if the data is already stored in memory. If the data is found in the cache, the application can quickly retrieve and return the data, reducing the need to query the persistent data store.
- *Claim check*: By separating data from the messaging flow, this pattern reduces the size of messages and supports a more cost-effective messaging solution.
- *Competing consumers*: This pattern efficiently handles items in a queue by applying distributed and concurrent processing. This design pattern optimizes costs by scaling that is based on queue depth and setting limits on maximum concurrent consumer instances.

- *Compute resource consolidation*: This pattern increases density and consolidates compute resources by combining multiple applications or components on shared infrastructure. It maximizes resource utilization, avoiding unused provisioned capacity and reducing costs.
- *Deployment stamps*: The use of deployment stamps provides several advantages, such as geo-distributing groups of devices, deploying new features to specific stamps, and observing cost per device. Deployment stamps allow for better scalability, fault tolerance, and efficient resource utilization.
- *Gateway offloading*: This pattern offloads request processing in a gateway device, redirecting costs from per-node resources to the gateway implementation. Using this design pattern can result in a lower cost of ownership in a centralized processing model.
- *Publisher/subscriber*: This pattern decouples components in an architecture, replacing direct communication with an intermediate message broker or event bus. It enables an event-driven approach and consumption-based billing, avoiding overprovisioning.
- *Queue-based load leveling*: The pattern buffers incoming requests or tasks in a queue. The buffering smooths out the workload and reduces the need for overprovisioning of resources to handle peak load. Incoming requests are processed asynchronously to reduce costs.
- *Sharding*: This pattern directs specific requests to a logical destination, allowing optimizations with data colocation. Sharding can lead to cost savings by using multiple instances of lower-spec compute or storage resources.
- *Static content hosting*: This pattern delivers static content efficiently by using a hosting platform designed for this purpose. It avoids the use of more expensive dynamic application hosts, optimizing resource utilization.
- *Throttling*: This pattern puts limits on the rate (rate limiting) or throughput of incoming requests to a resource or component. It helps inform cost modeling and can be tied directly to the business model of the application.
- *Valet key*: This pattern grants secure and exclusive access to a resource without involving more components, reducing the need for intermediary resources and improving efficiency.

Control supply

Defining an upper limit on the amount that you're willing to spend on a particular resource or service is one way to control supply. It's an important strategy for controlling costs and ensuring

that expenses don't exceed a certain level. Establish a budget and monitor the spending to ensure it stays within the defined amount. You can use cost management platforms, budget alerts, or tracking usage and spending patterns. Some services allow you to throttle supply and limit rates, and you should use those features where helpful.

Controlling supply refers to defining an upper limit on the amount that you're willing to spend on a particular resource or service. It's an important strategy because it helps control costs and ensures that expenses don't exceed a certain level. Establish a budget and monitor the spending to ensure it stays within the defined threshold. You can use cost management platforms, budget alerts, or tracking usage and spending patterns. Some services allow you to throttle supply and limit rates, and you should use those features where helpful.

> **Tradeoff**: Stricter limits might result in missed opportunities to scale when demand increases, potentially impacting user experience. It could cause shutdowns or unable to respond to load. It's important to strike a balance between cost optimization and ensuring that you have sufficient resources to meet your business needs.

Azure facilitation

Evaluating scale-out versus scale-up: Azure provides a test environment where you can deploy and test different scaling configurations. By using the actual workload data or proxy data, you can simulate real-world scenarios and measure the effects on costs. Azure offers tools and services for performance testing, load testing, and monitoring, which can help you evaluate the cost effectiveness of scale out versus scale up options.

Azure provides cost management recommendations through various tools and services, such as the Azure Advisor. These recommendations analyze your usage patterns, resource utilization, and scaling configurations to provide insights and suggestions for optimizing costs.

Azure Load Testing is a fully managed load-testing service that generates high-scale load. The service simulates traffic for your applications, regardless of where they're hosted. Developers, testers, and quality assurance (QA) engineers can use load testing to optimize application performance, scalability, or capacity.

Optimizing autoscaling: Many Azure compute services support deploying multiple identical instances, and rapidly tuning the scaling thresholds and policies. Azure provides autoscaling capabilities that allow you to automatically adjust the number of instances or resources based on workload demand. You can define scaling rules and thresholds to trigger scale-out or scale-in

actions. By using autoscaling, you can optimize resource allocation and cost efficiency by dynamically scaling resources based on actual demand.

Azure maintains a list of subscription and service limits. There's a general limit to the number of instances of a resource you can deploy in each resource group with some exceptions. For more information, see Resource instance limits per resource group.

Optimizing demand and supply: Azure Monitor provides insights into the performance and health of your applications and infrastructure. You can use Azure Monitor to monitor the load on your resources and analyze trends over time. By using metrics and logs collected by Azure Monitor, you can identify areas where scaling adjustments might be needed. This information can guide the refinement of your autoscaling policy to ensure it aligns with the nonfunctional requirements and cost optimization goals.

- *Offloading supply*: Azure has a modern cloud Content Delivery Network (CDN) called Azure Front Door and caching services (Azure Managed Redis and Azure HPC Cache). The CDN caches content closer to the end-users, reducing network latency and improving response times. Caching stores a copy of the data in front of the main data store, reducing the need for repeated requests to the backend. By using CDN and caching services, you can optimize performance and reduce the load on servers for potential cost savings.

- *Controlling supply*: Azure also allows you to set resource limits for your cloud workload. By defining resource limits, you can ensure that your workload stays within the allocated resources and avoid unnecessary costs. Azure provides various mechanisms for setting resource limits such as quotas, policies, and budget alerts. These mechanisms help you monitor and control resource usage.

  API Management can rate limit and throttle requests. Being able to throttle incoming requests is a key role of Azure API Management. Either by controlling the rate of requests or the total requests/data transferred, API Management allows API providers to protect their APIs from abuse and create value for different API product tiers.

Related links

- Scale the workload
- Azure Advisor Cost recommendations
- What is Azure Load Testing?
- Azure subscription and service limits, quotas, and constraints
- Resources not limited to 800 instances per resource group

- What is Azure Front Door?
- What is Azure Managed Redis?
- What is Azure HPC Cache?
- Advanced request throttling with Azure API Management

Cost Optimization checklist

Refer to the complete set of recommendations.

Cost Optimization checklist

Last updated on 11/15/2023